

CSP-J 入门级知识清单

知识范围、复杂度与实现检查

示例基线：C++14 (-O2 -std=c++14), 具体语言版本、时限和内存以当年比赛公告为准。

使用说明：逐项确认适用条件、正确实现和复杂度，并为关键结论准备边界数据或反例。

星号说明：带 * 的内容不属于入门级大纲的知识类别，属于提高级、NOI 级或补充内容；入门级类别下的常见实现、变体和应用不标星。

目录

1 入门级知识范围核对	2
2 C++ 语言与实现基础	3
3 基本问题求解与复杂度	4
4 字符串	5
5 数据结构、搜索与图论	6
6 动态规划	7
7 数学	8
8 * 数据范围与复杂度反推 (OI 视角)	10
9 * 空间复杂度与四档内存反推	11
10 * 提交与编译检查	12
11 *C++ UB 与实现风险	13
12 * 对拍流程	14
13 * 重点补充项	16
创作声明与许可	16

1 入门级知识范围核对

本节列出入门级需要掌握的基础知识与对应检查项。

知识模块	必须掌握的完整范围	检查要求
基础与环境	CPU、内存、I/O；位、字节与字；Windows/Linux 与文件目录操作；网络和 Internet 基本概念；计算机与 NOI 历史、活动规则；语言、编译、运行、IDE 与 g++。	☐ 把 512 MiB 换成字节和 bit；不用 IDE 说明源码怎样编译、运行和重定向。
程序基础	标识符、关键字、常量、变量、字符串、表达式；命名与作用；头文件、名字空间；编辑/编译/解释/调试；四类基本类型；输入输出、条件、循环；算术/关系/逻辑/自增/三目/位运算；数学库；模块化设计与流程图。	☐ 区分声明、初始化和赋值；把含循环、无解分支的流程图转成 C++14。
数组与函数	一维/多维数组；字符数组与 string；函数定义和调用、形参与实参、值/引用传参、作用域与递归。	☐ 计算多维数组字节数；读取含空格行；把复制 vector 的函数改成 const&。
复合类型	结构体、联合体、枚举；指针、指针数组访问、字符指针、结构体指针和引用。	☐ 用结构体和枚举描述搜索状态；判断指针/引用在函数返回和 vector 扩容后是否有效。
文件与 STL	文本/二进制文件、重定向和文件读写；min/max/swap/sort；stack/queue/list/vector。	☐ 说明文本读数与读取二进制表示的差别；解释 list 无随机访问且不能直接套 std::sort。
线性结构	单链表、双向链表、循环链表、栈和队列；链表 O(1) 删除的前提是已经持有正确节点。	☐ 画出删除双向链表中中间节点前后的四个指针变化。
简单树	树与二叉树的定义、性质、表示和存储；二叉树前序、中序、后序遍历。	☐ 对含空孩子的树写三种遍历，并解释一种遍历为何通常不能唯一还原树。
特殊树	完全二叉树及数组表示；哈夫曼树的构造与编码；二叉搜索树的定义和构造。	☐ 区分完全/满/BST；用递增插入说明普通 BST 可退化为 $O(n)$ 。
简单图	图的点、边、方向、度、路径、连通等概念；邻接矩阵和邻接表。	☐ 比较 $O(V^2)$ 与 $O(V+E)$ 空间，并正确处理无向边的两条弧。
算法与策略	算法概念；自然语言、流程图、伪代码描述；枚举、模拟、贪心、递推、递归、二分、倍增；前缀和与差分。	☐ 为贪心写证明、为二分写单调谓词、为枚举证明不重不漏。
高精度	高精度加法、减法、乘法；高精度整数除单精度整数的商和余数。	☐ 用 1000-1、999*999、1000/7 检查借位、进位和余数。
排序	排序概念；冒泡、选择、插入、计数排序；稳定性、时间和额外空间。	☐ 在重复键数据上判断四种排序的稳定性；说明计数排序依赖值域。
搜索与图遍历	DFS、BFS；图的深度/广度优先遍历；Flood Fill。	☐ 给带权图反例说明普通 BFS 不能求一般最短路；在含环图中保证状态只访问一次。
动态规划	DP 基本思想；简单一维、简单背包、简单区间 DP。	☐ 分别写状态、初值、转移顺序和答案位置；不可达状态不能默认设 0。
数与数论	数集和四则运算；二/八/十/十六进制；初中代数/几何；整除、因数、倍数、指数、质合数、取整、模、唯一分解、欧几里得、埃氏筛和线性筛。	☐ 测试 0/1/2 与平方数；区分 C++ 负数除法和数学 floor。
组合与其他	集合；加法/乘法原理；排列、组合、杨辉三角；ASCII 码。	☐ 用小枚举检查计数是否重复；不用魔法数字完成数字字符转换和大小写判断。

2 C++ 语言与实现基础

知识点	适用条件	检查要求
整数类型范围与溢出	用于可能产生大整数的运算。类型选择须覆盖输入、答案和中间结果；扩大目标变量类型不能修复已经发生的窄类型溢出。	实现受限乘方：输入 a, b ($a, b \leq 10^9$)，若 $a^b > 10^9$ 输出 -1 ；扩型和溢出检查须在乘法之前完成。
整数除法与取模	整除、向上取整、余数状态和二进制拆分。C++14 的有符号整数除法向零取整；负数取模的符号也要按语言规则判断。	给定 $a = -7, b = 3$ ，输出 C++14 下 a/b 和 $a\%b$ ，解释结果差异。
浮点误差与精确比较	百分比、 $\sqrt{\Delta}$ 和分数比较优先使用整数。double 无法精确表示 0.1。	实现整数二分开方 $\text{isqrt}(n)$ ，不使用 sqrt ；用 $n = 10^{18}$ 验证。
数组、字符、string	读含空格的句子、解析 IP、统计字符频次。cin >> s 会跳过空格。	读一整行（可能含空格和数字），统计字母和数字各出现几次。
函数、引用、指针	传递大型对象（如 vector、邻接表）时避免不必要复制；小型标量按值传递未必更慢。	将 <code>void f(vector<int> v)</code> 改为 <code>void f(const vector<int>& v)</code> ，解释快的原因。
递归与调用栈	递归适用于树遍历、分治排序和表达式求值；深度较大时可能耗尽调用栈。栈帧大小和栈上限取决于编译器、平台和局部变量，须按目标环境核对。	分别实现递归 DFS 和显式栈 DFS，并根据最大深度判断是否需要迭代实现。
输入输出与文件 IO	输入输出量大时可用 <code>std::ios::sync_with_stdio(false)</code> 与 <code>std::cin.tie(nullptr)</code> ；是否超时取决于数据量、输出量和时限，不存在固定的 n 阈值。	用 <code>freopen</code> 重定向的例子。解释 <code>cin >></code> 后接 <code>getline</code> 读到空串的原因。
vector、queue、*deque	邻接表、BFS 队列、模拟队列。分清各容器适用场景。	手写循环队列（push/pop/front，用数组），说出 <code>std::queue</code> 底层是 deque。
*STL map、set、heap	适用于值域较大、需要有序遍历或需要反复取得最小距离的情况。priority_queue 默认按最大元素优先。	实现最小优先队列，并据此完成 Dijkstra。
运算符优先级、结合性与短路	写表达式解析或混合多个运算符时，不能靠猜优先级。位运算 $\& \mid \sim$ 优先级低于 $== \neq$ ；逻辑 $\&\& \mid\mid$ 有短路语义；赋值 = 优先级极低。括号是最便宜的正确性保障。	给出 <code>int a=2,b=3,c=4;</code> ，不加括号解释 <code>a&b==2</code> 和 <code>a b&c</code> 的实际求值顺序，并说明为何 <code>if(x&1==0)</code> 不判断 x 最低位是否为 0。
C++14 版本与编译选项	本文以 <code>-std=c++14</code> 为基线； <code>std::gcd</code> 、结构化绑定、 <code>if constexpr</code> 属于 C++17，不能依赖编译器扩展。	用 <code>g++ -std=c++14 -O2</code> 编译最近写的一份代码；静态链接、编译器和内存限制以比赛环境为准。

3 基本问题求解与复杂度

知识点	适用条件	检查要求
模拟	适用于状态转移和操作顺序已由题面明确给出的情形。实现时须完整保留条件和执行顺序。	将题面描述拆成有先后约束的单步操作，并标明每步修改的状态。
枚举	数据规模小的时候枚举所有可能性；剪枝只能降低实际运行量，不能改变最坏复杂度。	写 N 个数的子集枚举（位运算版和递归 DFS 版各一）。
递归、* 分治、显式栈	适用于分治排序、树遍历和括号解析；最大深度较大时应采用迭代实现。	分别实现递归版和迭代版归并排序，说明递归深度与栈空间差异。
贪心	适用于局部选择可通过交换论证、支配关系或不变量推出全局最优的情形。	对一个未经证明的局部最优策略构造反例，再补充成立条件或改用其他算法。
* 贡献计算	把总答案拆成每个元素、位置、边或区间的独立贡献，常能把两重枚举改成一次统计、前缀和或频次维护。	求所有子数组元素和时，推导 a_i 被多少个子数组包含；再用小规模暴力对拍贡献式。
前缀和、差分	前缀和适合频繁区间聚合查询；差分适合区间加减，最后扫描一次还原。两者的 $O(1)$ 是针对特定操作，不是所有区间操作。	实现数组区间加 1、最后输出所有位置的值，要求 $O(N + Q)$ 。
二分查找与二分答案	有序数组中找边界，或答案范围大（如 10^9 ）且判定具有单调性；先区分找任意值、第一处真和最后一处真。	分别写 <code>lower_bound</code> 、第一处可行和最后处可行模板，处理空区间、重复值和中点溢出。
* 三分查找	目标函数在区间内单峰或单谷，且能比较两个位置的函数值时使用；不满足单峰性不能套模板。	在整数区间上写三分并处理平台、端点和最终枚举；说明实数三分的精度停止条件。
双指针、滑动窗口、* 单调队列	时间窗（24 小时内乘客）、价格窗（45 分钟票）、坐标窗口：窗口单向滑动。	证明“每个元素最多出入队一次”。举看似两重循环、实为 $O(N)$ 的例子。
* 离散化	值域为 10^9 而有效值只有 10^5 个时，按原值域分配数组会超出常见内存限制。	整数数组排序去重后用 <code>lower_bound</code> 做压缩映射，写出完整代码。
倍增	树上跳 k 级祖先、数组跳区间。把 k 按二进制拆位。	给出 <code>up[j][v]</code> 的两种含义：从 v 跳 2^j 步到哪，以及转移方程。
* 复杂度分析	统计所有循环和数据结构操作的总执行次数。队列和栈常按每个元素的总进出次数进行摊销分析。	给一段同时含 <code>while</code> 弹队列和 <code>for</code> 扫描的代码，判断 $O(N)$ 还是 $O(N^2)$ 。
* 测试点与部分分	若题面提供测试点或分组限制，小规模、全相等、 $k=0$ 等条件可能对应部分分；没有测试点表时也要根据约束自行分层。	根据题目约束与测试点，分别标注可用直接算法和需要优化的子任务。
* 边界数据	补充样例未覆盖的最小值、最大值、全相等、无解和等号边界。	为一段排序代码造 10 组数据，覆盖升序、降序、全相等、含负数、单元素。
* 正确性说明	为核心状态和循环写出不变量，核对初始化、更新顺序和结束条件。	说明核心循环执行前后关键变量的含义，并据此检查边界。

4 字符串

知识点	适用条件	检查要求
字符串解析与格式转化	进制转换（读入 k 进制字符串转整数、或整数转 k 进制输出）、高精度读入（逐字符转数字数组）、IPv4/端口格式校验（a.b.c.d:e 分五段，禁前导零、检查值域）。核心是逐字符处理、手工分段、用整数数组承接。	写 k 进制转十进制：逐位累加 $\text{ans} = \text{ans} * k + \text{digit}$ 。写高精度读入：把长度 $\leq 10^6$ 的数字串转成 <code>vector<int></code> 。写 IPv4 解析：严格拒绝空段、前导零（如 01）、越界（ $0 \sim 255$ 和 $0 \sim 65535$ ）和多余字符。
字符串直接扫描	当 $n, q \leq 1000$ 且 $nq \leq 5 \times 10^8$ 时，可优先检查逐对比较子串或逐查询扫描全表；是否覆盖全部测试点由完整约束决定。	实现后缀查询：每次扫描全部书号，用 $\text{code} \% 10^{\text{len}} == \text{need}$ 判断后缀，并写出 $O(nq)$ 时间和 $O(1)$ 额外空间。
回文判断、子串与字符频次	回文可用 $O(N)$ 双指针或构造候选年份；子串计数可用滑动窗口或枚举起点；字符频次可用固定大小数组统计。	写回文日期题：枚举 4 位年份按回文规则构造 8 位日期，验证月日合法性后判断是否在输入区间内。再写标题统计题：逐字符分类计数字母和数字，注意空格和换行。说明“枚举年份构造”和“逐日推进”的复杂度差异。
*Trie（前缀树）	大量字符串、前缀查询；即使不直接裸考，也可能作为字典、前缀统计或状态结构的组件。	用数组写 Trie，插入 cat/car/dog，查前缀 ca 的词数。
* 字符串哈希	快速比较子串、判断树镜像。预处理 $O(L)$ 后每次查询 $O(1)$ ，但哈希存在碰撞风险。	约定区间为半开区间 $[l, r)$ ，推导 $\text{hash}(l, r) = H[r] - H[l] \times \text{base}^{r-l}$ ；说明模数、底数和双哈希如何降低风险。

5 数据结构、搜索与图论

知识点	适用条件	检查要求
栈、队列、* 双端队列	BFS (队列)、括号匹配 (栈)、滑动窗口最值 (单调队列)。元素最多进出一次是摊销核心。	手写循环队列 (数组实现，不用 STL)，解释判空判满为什么要留一个空位。
* 堆与优先队列	Dijkstra 取最小距离、事件模拟最早发生。默认大根堆，取最小值得改比较器。	写 Dijkstra：为什么同一点可能入堆多次？旧的必须跳过？
* 集合、映射、哈希	值域小用数组；需要有序遍历或最坏复杂度保证时用 map；能接受期望 $O(1)$ 且注意退化风险时再用 unordered_map。	构造一组让 unordered_map 退化成 $O(N)$ 的输入。
* 多重计数与值映射	重复值按频次、类别或出现位置统计；值域小用频次数组，值域大用 map、哈希或离散化，不能把“不同值个数”和“元素总数”混为一谈。	对 [3, 1, 3, 2, 3] 同时输出每个值的频次、出现位置和不同值个数，比较数组、map 与离散化三种写法。
* 并查集	只问“两个点是否连通”，不问路径。	手写 find (路径压缩) 和 merge (按大小合并)，说明为什么路径压缩不破坏正确性。
树与二叉树	用于树的遍历、子树大小和镜像判断。须统一空子树表示；深度较大时改为迭代实现。	用数组存二叉树，写先序和后序遍历，禁止递归。
图存储与遍历	稀疏图或点数较大时用邻接表；很小的稠密图才考虑邻接矩阵。有向图、无向图建图不同。	给一张 N 点 M 边无向图，用邻接表，BFS 求 1 到各点最短步数。
链表与 * 有序块删除	反复取走序列元素、合并相邻同类块。链表在已经定位节点时删除可为 $O(1)$ ，查找节点和维护块边界仍可能成为瓶颈。	模拟 0011100：每轮取每个块最左元素，写出每轮输出。
* 树状数组	单点修改、前缀和查询。比线段树短。稳定排序里的排名也能算。	手写 add 和 sum，说明 lowbit。在 [3, 1, 3] 上模拟全部更新。
DFS、BFS	可达性、无权最短路、连通块。标记要在入队时做，而非出队时。	在有环图上逐层写出 BFS 入队顺序，解释为何标记时机重要。
* Dijkstra 与 0-1 BFS	边权非负时可用 Dijkstra；边权仅为 0, 1 时可用 deque 做 0-1 BFS。一般非负权不应误写成普通 BFS。	用天平类比松弛：为什么非负权下当前弹出的最小有效距离可以定型？旧堆项为什么必须跳过？
* 状态图 (奇偶/资源维)	“恰好 L 步”、“最多 k 次魔法”、“必须模 k 时刻”：加维度进状态。	无向图 s 到 t 恰好 L 步不能只看最短路；还要考虑连通分量、二分图奇偶性和是否存在可往返的边或奇环。构造二分图和含奇环的图，分别判断恰好 L 步可达性，说明为什么“来回 2 步”只是常用的扩展手段，不是完整判定。

6 动态规划

知识点	适用条件	检查要求
状态定义、转移、初始化	写 DP 先定状态： $dp[i]$ 是“以 i 结尾”还是“前 i 个中选”。初始化不能默认 0；“恰好”通常用 $-\infty$ 标不可达，“至多”或空集合法可用 0。	写“恰好 n 根拼数字”的 DP，解释不可达为何不能初始化为 0。再写“网格取数”：允许上下右移动、格权可为负，全负网格为何 DP 初值不能是 0 而要用 $-\infty$ 。
DP 识别：从暴力到 DP	题面要求“最多/最少/方案数”、数据 $n \leq 500$ 但 2^n 枚举不可能时，优先考虑 DP。核心信号：存在重叠子问题（同一状态被多次计算）、存在最优子结构（大问题的最优解包含子问题的最优解）。	给摆渡车问题： n 个同学在不同时刻到达，发车间隔固定为 m 。先写按到达时间分组的 $O(2^n)$ 暴力枚举发车方案，再指出“前 i 个人的最小等待和”只依赖前面状态，写出 $dp[i]$ 转移。对比暴力与 DP 的复杂度差距。
背包 DP	选若干物品使和满足某条件。对正权容量的一维 DP，0/1 背包通常从大到小，完全背包通常从小到大；计数、负权或其他转移要重新证明顺序。	写 0/1 背包，解释容量循环方向，再改成完全背包。
* 网格 DP / 带预算 DP	网格上单向走、带预算 k 选点。1000 × 1000 时不能 $O(N^3)$ 。预算维通常来自题面的 $k \leq 100$ 这类小参数信号。	3 × 4 网格手动跑 DP，只准右/下，取最大和。再写出状态为 $dp[i][j][used]$ 的三维带预算版。
线性 DP 与区间 DP 入门	线性 DP ($dp[i]$ 只依赖前若干项) 是 CSP-J 最常考的 DP 形态。区间 DP ($dp[l][r]$ 依赖子区间) 在回文、括号、取数等场景出现。两者都可用 $O(N^2)$ 拿部分分，再优化。	写最长上升子序列 $O(N^2)$ 版。再写“网格取数”的按列 DP：每列从上到下、从下到上各扫一次，解释为何不能在同一列内来回走（违反不能重复经过的约束）。
* 单调队列优化 DP	转移从滑动窗口取 max/min 时，直接枚举多 $O(W)$ ，单调队列压到 $O(N)$ 。	数组 [5, 1, 4, 4, 2]，窗口 3，逐步写出队列变化。
* 方案恢复与滚动数组	只问最优值时可滚动；要恢复方案时通常保存父指针或决策，不能把必要信息一起覆盖。也可用重算等方法换空间。	在背包 DP 上加 choice 数组，输出最优值的同时输出选了哪些物品。
* DP 维度选择与部分分信号	从题面约束倒推 DP 维数： $n \leq 500$ 且 $k \leq 100 \rightarrow$ 二维 $dp[i][k]$ （带预算的点列选择）； $m \leq 100$ 且 $n \leq 500 \rightarrow dp[i]$ 一维 + 余数优化（固定间隔发车）；网格 1000 × 1000 $\rightarrow$ 按列滚动（网格取数）；值域小 ($a_i \leq 5000$) $\rightarrow$ 以值域为容量做背包 DP（选木棍拼多边形）。	给一道新题的数据范围，能写出至少两种可能的 DP 状态设计，并估算各自的时间/空间复杂度，选出更可行的方案。

7 数学

知识点	适用条件	检查要求
基本运算与边界	max/min、绝对值。abs(INT_MIN) 的结果无法由 int 表示，取负或调用对应重载可能触发未定义行为。	不用分支和库函数写整数绝对值。说出 abs(INT_MIN) 的后果。
位运算	判奇偶、提取二进制位、异或 ($a \oplus a = 0$)、lowbit、枚举子集 (for(int s=mask; s; s=(s-1)&mask))。	写代码：对正整数只用位运算判 2 的幂，不准用循环；用位运算枚举 $\{a, b, c\}$ 的所有非空子集。
质数判定与埃氏筛	单个数判质数可试除到 $\sqrt{n}$ ；需要批量筛一段连续范围的质数时用埃氏筛， $O(N \log \log N)$ 。	手写埃氏筛求 $[1, 100]$ 的质数表，说明为什么内层从 i^2 开始、为什么只用 $\leq \sqrt{N}$ 的质数去筛。
线性筛（欧拉筛）与最小质因子	要求 $O(N)$ 或需要每个数的最小质因子时用线性筛。核心：每个合数只被其最小质因子筛掉一次。可顺势维护 $lp[x]$ 用于 $O(\log x)$ 因数分解或积性函数递推。	写线性筛同时维护 prime[] 和 $lp[x]$ ，在纸上模拟 $\{2 \dots 30\}$ ：指出 12 被 2 筛、15 被 3 筛、25 被 5 筛，说明判断 $i \% prime[j] == 0$ 后 break 的原因。
gcd 与 lcm	约分、分数化简。std::gcd 是 C++17 才有，C++14 自己写；计算 lcm 时先除后乘以降低溢出风险。	手算 gcd(12345, 67890)，写每一步余数。
* 模运算与快速幂	答案模 $10^9 + 7$ 或 998244353；指数很大时用快速幂把乘法次数从 $O(n)$ 降到 $O(\log n)$ 。	写快速幂递归和迭代版各一，验证 $3^{10} \bmod 7 = 4$ ，并说明二分拆指数的正确性。
* 贡献计算	不枚举所有子结构，改为计算每个元素在最终答案中被计入的次数。常用于“所有子数组的 max/min/sum 之和”“每个节点对多少条路径有贡献”等聚合题。核心公式： $ans = \sum_i val_i \times count_i。$	给定数组 $[2, 1, 3]$ ，用单调栈求每个元素作为最大值的左右边界，计算以它为最大值的子数组个数，验证所有子数组最大值之和是否为 $2 \cdot 1 + 1 \cdot 1 + 3 \cdot 3 = 12$ 。手算与代码一致。
组合数学基础	先判断分类是否互斥且完备，再使用加法原理；分步完成且步骤相互独立时使用乘法原理。确定计数对象与分类后，再判断是否需要 DP。	对“选一个人”与“先选组别再选组内成员”各写一种计数式，指出重复计数和漏计数的位置。
排列、组合与重复计数	区分排列 $P(n, k)$ 、组合 $C(n, k)$ 、可重复选择 (n^k) 和相同元素排列（多重集排列公式）。常与全排列、子集、频次分配结合。	手算 $C(10, 3)$ ，分别解释“从 10 人中选 3 人排队”和“选 3 人开会”的答案为何不同。给 $\{a, a, b\}$ 写出所有不同排列。
组合递推与杨辉三角	$C(n, k) = C(n-1, k-1) + C(n-1, k)$ ，边界为 $C(n, 0) = C(n, n) = 1$ ；大量询问时预处理、取模、滚动空间。	写杨辉三角前 5 行，验证一行的和为 2^n ，并说明组合数取模时不能随意使用实数除法（要用递推或乘法逆元）。
* 多重计数、映射与双射	直接计数困难时，建立原问题与另一个易计数集合的双射（一一对应）；或按某个特征分类后对每类分别计数再相加；也可用“计数 $\times$ 量 = 总量”的双视角技巧。	用隔板法（Stars and Bars）解释“长度为 n 、和为 S 的非负整数序列个数”为 $C(S+n-1, n-1)$ ，手算 $n=3, S=4$ 的结果并列验证。举例说明如何把“子数组和为 K ”映射为前缀差对 (i, j) 满足 $pre[i] - pre[j] = K$ 。
一元幂方程与单调方程	求 $x^k = A$ 、 $ax + b = c$ 或单调函数的整数根时，利用定义域、单调性和二分；一般高次多项式没有可直接套用的通用初等公式。	写整数二分求最大的 x 使 $x^k \leq A$ ，避免乘法溢出并回代检查；比较整数答案和实数近似的处理差异。
一元二次方程求解	$ax^2 + bx + c = 0$ ($a \neq 0$)：先算 $\Delta = b^2 - 4ac$ 。 $\Delta < 0$ 无实根； $\Delta \geq 0$ 继续。整数根条件： Δ 为完全平方数且 $-b \pm \sqrt{\Delta}$ 被 $2a$ 整除。非整数需精确输出：约分有理数，或提取 $\sqrt{\Delta}$ 的平方因子写成规范根式 $\frac{p \pm q\sqrt{s}}{r}$ ，保证 $\gcd(p, q, r) = 1$ 且 s 无平方因子。	写程序对 $x^2 - 3x + 1 = 0$ 输出两个根的规范形式（含 $\sqrt{5}$ ），并处理 $a < 0$ 、 b^2 溢出（用 long long）、 Δ 非平方、分母约分和较大根选择。对照历年真题中 RSA 解密和一元二次方程输出题验证。
整数开方	需要精确整数答案或判完全平方数时，优先用整数运算并验证；不能把浮点 sqrt 的结果未经检查直接转整数。	写整数二分开方，避免 mid*mid 溢出，验证 $\lfloor \sqrt{10^{18}} \rfloor$ 。

知识点	适用条件	检查要求
二分与 * 三分	二分用于单调函数求零点或阈值：整数域上用闭区间不变量防死循环，实数域按精度或固定次数迭代。三分用于单峰/单谷函数求极值：每轮比较 $f(m_1)$ 与 $f(m_2)$ 扔掉极值不在的 $1/3$ 区间；离散域上区间缩小到 ≤ 3 后枚举。与第 2 节“二分答案”的差异：这里面对的是具体函数 $f(x)$ ，不要求构造判定谓词。	写整数三分在 $[0, 10]$ 上找 $f(x) = -(x-3)^2 + 5$ 的最大值，先用枚举验证极值位置，再解释三分为何每次扔掉 $1/3$ 区间，并说明整数三分何时需要转枚举收尾。

8 * 数据范围与复杂度反推 (OI 视角)

将总操作数展开为包含全部独立参数的公式，再与 $B = 5 \times 10^8$ 比较。表中的 $\log$ 均按 $\log_2$ ；边界取满足公式不超过 B 的最大整数，不计常数。 $n = 5000$ 时， $O(n^2)$ 为 2.5×10^7 ，符合本时间口径；空间另行计算。

复杂度	最大整数规模	边界校验	常见候选
$O(n^n)$	$n \leq 9$	$9^9 = 387,420,489$, $10^{10} > B$	每个位置有 n 种选择
$O(n!)$	$n \leq 12$	$12! = 479,001,600$, $13! > B$	全排列
$O(3^n)$	$n \leq 18$	$3^{18} = 387,420,489$	三进制状态
$O(n^2 2^n)$	$n \leq 20$	$20^2 \cdot 2^{20} = 419,430,400$	状压双层转移
$O(n 2^n)$	$n \leq 24$	$24 \cdot 2^{24} = 402,653,184$	子集加末项
$O(2^n)$	$n \leq 28$	$2^{28} = 268,435,456$, $2^{29} > B$	子集枚举
$O(n^5)$	$n \leq 54$	$54^5 = 459,165,024$	五重枚举
$O(n^4)$	$n \leq 149$	$149^4 = 492,884,401$	四重枚举
$O(n^3)$	$n \leq 793$	$793^3 = 498,677,257$	三重枚举
$O(n^2 \log n)$	$n \leq 6294$	$6294^2 \log_2 6294 \approx 4.999 \times 10^8$	二维状态加对数操作
$O(n^2)$	$n \leq 22360$	$22360^2 = 499,969,600$	点对、二维 DP
$O(n\sqrt{n})$	$n \leq 629960$	$n\sqrt{n} \approx 4.99999 \times 10^8$	分块、因数枚举
$O(n \log n)$	$n \leq 20,580,553$	$n \log_2 n \approx 4.99999975 \times 10^8$	排序、堆、有序结构
$O(n)$	$n \leq 500,000,000$	$n = B$	扫描、前缀、线性 DP
$O(\log n)/O(1)$	常用整数范围	仍须乘调用次数并计输入输出	二分定位、公式

多参数直接展开： T 组 $O(n^2)$ 写成 Tn^2 ； Q 次长度 n 的扫描写成 Qn ；图写 $V + E$ 或 $(V + E) \log V$ ；背包写 nW ；网格写 rc ；状态图写 $V \cdot K$ 个状态。结果 $\leq 5 \times 10^8$ 即通过本时间口径，随后单独检查内存。

9 * 空间复杂度与四档内存反推

使用 $1 \text{ MiB} = 2^{20} \text{ B}$ 。下表只是一块连续载荷占满限制的数学上限；程序映像、其他数组、容器容量、分配器、临时对象和调用栈全部计入总内存，实装时必须逐项求和。按常见 64 位 GCC 环境估算：
char/bool=1 B、int=4 B、long long/double=8 B；结构体用目标环境 sizeof 核准。

限制	总字节/1 B 元素	int 元素	8 B 元素	压位布尔 bit
128 MiB	134,217,728	33,554,432	16,777,216	1,073,741,824
256 MiB	268,435,456	67,108,864	33,554,432	2,147,483,648
512 MiB	536,870,912	134,217,728	67,108,864	4,294,967,296
1024 MiB	1,073,741,824	268,435,456	134,217,728	8,589,934,592

单一方阵理论边长	128 MiB	256 MiB	512 MiB	1024 MiB
int a[n][n]	$n \leq 5792$	$n \leq 8192$	$n \leq 11585$	$n \leq 16384$
long long a[n][n]	$n \leq 4096$	$n \leq 5792$	$n \leq 8192$	$n \leq 11585$

校准：int dp[5000][5000] 是 100,000,000 B，约 95.37 MiB；两份约 190.73 MiB。 $n = 20000$ 的 $O(n^2)$ 时间是 4×10^8 ，满足时间预算，但 int[n][n] 约 1525.88 MiB，1024 MiB 仍放不下。

结构/算法	渐进空间	必须展开的峰值
前缀和/差分	$O(n)$	一个 long long 数组约 $8(n+1) \text{ B}$ ；原数组若仍存活要相加。
BFS/DFS	$O(V+E)$	图 + 标记/距离 + 队列/显式栈；递归另有 $O(\text{height})$ 调用栈。
邻接矩阵	$O(V^2)$	int 矩阵 $4V^2 \text{ B}$ 。
数组邻接表	$O(V+E)$	每弧三个 int 约 $12E + 4V \text{ B}$ ；无向 m 边存两倍弧，约 $24m + 4V \text{ B}$ 。
Dijkstra	$O(V+E)$	图、距离和懒删除优先队列同时存活；旧状态也计峰值。
归并排序	$O(n)$	原数组 + $\text{sizeof}(T) \cdot n$ 临时数组 + $O(\log n)$ 栈。
数组 Trie	$O(L \Sigma)$	$4 \cdot \text{maxNode} \cdot \text{alphabet} \text{ B}$ ；节点上界是总字符数加 1。
背包滚动	$O(W)$	一个 int 层 $4(W+1) \text{ B}$ ；保留全部 n 层则为 $O(nW)$ 。
区间/二维 DP	$O(n^2)$	long long 表为 $8n^2 \text{ B}$ ；时间可行不表示表能开。
map/set/哈希	$O(n)$	结点、指针、桶、对齐、负载因子和扩容峰值均超过裸值载荷。

- 列出所有同时存活的大对象并求和，不仅计算最大的单个对象。
- 分别在 128/256/512/1024 MiB 限制下判断方案，并指出首个超限对象。
- 将无向边的两条弧、容器冗余、递归栈和临时数组计入峰值。
- 判断 $O(n^2)$ 方案由时间还是空间先限制，并列操作数和 MiB。

10 * 提交与编译检查

本节条目参考《常见错误文档表》中整理的实际代码错误。文件名、编译选项和判定结果以当年规则及题面为准。

检查位置	常见错误	检查要求	可能结果
文件读写	题目要求文件 I/O 时，文件名或扩展名错误；输入输出文件写反；重定向被注释；使用本机绝对路径	按题面逐字核对文件名；输入使用读取模式并重定向到标准输入，输出使用写入模式并重定向到标准输出；不得保留本机路径	无输出、读入失败、零分
入口与头文件	main、include 或 namespace 拼写错误；头文件缺失；提交了错误语言格式	使用 C++14 编译最终提交文件，确认只有一个有效的 main，所用接口均有对应头文件	CE
字符串字面量	文件名等字符串误用单引号；字符串拼写与大小写不一致	字符使用单引号，字符串使用双引号；需要精确匹配的文本逐字符核对	CE、读写失败
调试清理	删除调试输出时破坏括号、分号或条件结构；遗留额外输出	清理后重新格式化并完整编译；用样例核对输出中没有提示语、日志或多余空格行	CE、WA
粘贴与文件内容	重复粘贴函数或完整程序；把样例输入、大段表格或无关文本写入源码	搜索重复定义；检查提交文件类型和大小；源程序不得超过当年规定上限	CE、提交失败
算法完整性	固定输出样例、按输入分支输出预置答案、随机输出或遗漏核心算法	使用未见过的合法小数据验证；确认输出由输入和算法状态计算得到	WA、零分
平台接口	使用 system()、本机专用命令或依赖本地目录结构	仅使用比赛环境允许的标准接口；在规定编译环境运行最终版本	CE、RE、无输出
初始化与多测	局部变量未初始化；数组或容器未在每组数据前复位	开启编译警告；逐项检查初值、不可达标记及多组数据的清空范围	WA、UB

- 最终文件能以规定的 C++14 选项独立编译。
- 文件读写仅在题面要求时启用，名称、扩展名和读写方向完全一致。
- 已删除调试输出、固定答案、本机路径和平台专用调用。
- 已核对提交文件、语言类型、源程序大小和多组数据初始化。

11 *C++ UB 与实现风险

种类	风险	错误示例	可能结果	修正
UB	有符号溢出	<code>int x=2000000000; x+=x;</code>	结果不可依赖	使用 <code>long long</code> 或先判 <code>x>LIMIT-a</code>
UB	左移溢出	<code>1<<31</code> (32 位 <code>int</code>)	出人意料的值	<code>1LL<<31</code> 或 <code>1u<<31</code>
UB	数组越界	<code>int a[10]; a[10]=0;</code>	覆盖相邻变量、段错误	每次访问前检查下标
UB	整数除零	<code>int x=a/b; (b 可能为 0)</code>	运行时未定义	分母可能为零时先判
UB	未初始化变量	<code>int x; if(x){...}</code>	结果随机	定义时赋初值
UB	迭代器失效	<code>v.erase(it); it++;</code>	跳过元素或崩溃	<code>it=v.erase(it)</code>
UB	严格弱序不满足	<code>sort(..., [](int a,int b){return a<=b;})</code>	排序崩溃	改 <code>return a<b;</code>
UB	悬空引用	<code>auto& r=*v.end();</code>	非法内存	不对 <code>end()</code> 解引用
资源	调用栈耗尽	<code>void f(){f();}</code>	RE / SIGSEGV	限制深度或改为迭代
UB	C++14 求值顺序	<code>arr[i]=i++;</code>	未定义行为 (C++14)	拆两行, 先保存旧值
UB	int 乘后赋 long long	<code>long long x=a*b;</code>	先在 <code>int</code> 中溢出	<code>1LL*a*b</code>
数值	浮点判等	<code>double a,b; if(a==b)</code>	相等判不出	<code>fabs(a-b)<eps</code> 或改整数
性能	哈希退化	<code>unordered_map</code> 发生集中冲突	单次操作退化为 $O(N)$	仅将 $O(1)$ 写作期望复杂度
UB	基类析构非虚	<code>Base*p=new Derived; delete p;</code>	通过基类删除是未定义行为	基类析构函数声明为 <code>virtual</code>

分类说明：UB 不具有可依赖结果；未指定行为表示标准未规定具体选择；数值风险表示程序合法但数值可能错误；性能问题表示行为确定但复杂度可能超限。

12 * 对拍流程

对拍由定义式暴力程序、候选程序、数据生成器和结果比较脚本组成。

三个独立程序

1. `brute.cpp`：按定义实现并优先保证正确；生成数据的规模须与其复杂度相匹配。
2. `sol.cpp`：待提交的候选程序。
3. `gen.cpp`：生成合法的小规模数据；以测试编号作为种子，保证失败用例可复现。

分别编译

任一源文件修改后均须重新编译：

```
g++ -std=c++14 -O2 -o brute brute.cpp
g++ -std=c++14 -O2 -o sol sol.cpp
g++ -std=c++14 -O2 -o gen gen.cpp
```

对拍脚本 `run_test.sh`

```
#!/bin/bash
for i in $(seq 1 1000); do
    ./gen "$i" > in.txt
    ./brute < in.txt > ans.txt
    ./sol < in.txt > out.txt
    if ! diff -q ans.txt out.txt > /dev/null; then
        echo "WA on test $i"; cp in.txt hack.in; exit 1
    fi
done
echo "All passed"
```

流程：gen 生成输入 → brute 生成标准输出 → sol 生成候选输出 → diff 比较结果。出现差异时保存输入到 `hack.in` 并停止；对可能不终止的程序设置运行超时。

生成器模板 `gen.cpp`

```
#include <cstdlib>
#include <iostream>
#include <random>
int main(int argc, char **argv) {
    unsigned seed = argc > 1
        ? static_cast<unsigned>(std::strtoul(argv[1], nullptr, 10))
        : 42u; // 无参数时仍可复现
    std::mt19937 rng(seed);
    std::uniform_int_distribution<int> n_dist(1, 10);
    std::uniform_int_distribution<int> a_dist(0, 99);
    int n = n_dist(rng);
    std::cout << n << "\n";
    for (int i = 0; i < n; i++)
        std::cout << a_dist(rng)
            << (i+1 == n ? '\n' : ' ');
    return 0;
}
```

模板仅演示可复现的随机种子。实际测试还须按题意加入极值、重复值、结构化数据和无解数据，均匀随机数据不能替代这些用例。

边界用例

- 最小规模 ($n = 1$) 和最大规模 (但仅能暴力跑的范围)
- 全相等 / 全 0 / 全 1
- 已升序 / 已降序
- 无解输入 (-1 或 NO 的情况)
- 紧贴边界 ($n = 10^9$ 触顶、模数边界等)

检查项

- ☐ 能在限定时间内完成暴力程序、候选程序、生成器和比较脚本。
- ☐ 生成器使用固定种子，并将首个差异输入保存到 `hack.in`。
- ☐ 出现 WA 时，能够分别核对生成器、暴力程序、候选程序和比较脚本。

13 * 重点补充项

- 历年 CSP-J/NOIP 普及组真题多次考查模拟和扫描，同时改变语言边界、整数范围与输入格式。
- **需要重点核对的内容：**
 1. 字符串区间哈希：能够预处理前缀哈希，并按统一区间定义比较子串。
 2. 数学与计数：能够使用贡献计算、映射或双射；区分排列与组合、加法原理与乘法原理；完成判别式与根式规范化。
 3. 复杂度反推：在实现前将全部参数代入时间预算，并独立核算峰值内存。
 4. 对拍：准备定义式暴力、可复现生成器、边界用例和首个失败用例。
- 线性筛、二分三分、Trie、区间哈希、树状数组、单调队列、状态图未必以裸模板出现，但常作为综合题的一部分。

创作声明与许可

除另有说明的第三方内容外，本文由 scmmm 原创。本文以 [Creative Commons 署名-非商业性使用 4.0 国际版 \(CC BY-NC 4.0\)](#) 发布。

在注明原作者为 scmmm、保留本许可声明和许可证链接，并标明修改内容的前提下，允许非商业性转载、分享与修改。任何商业性质的使用、转载、修改、发行，或将本文用于商业产品、课程或服务，均须事先获得 scmmm 的授权。